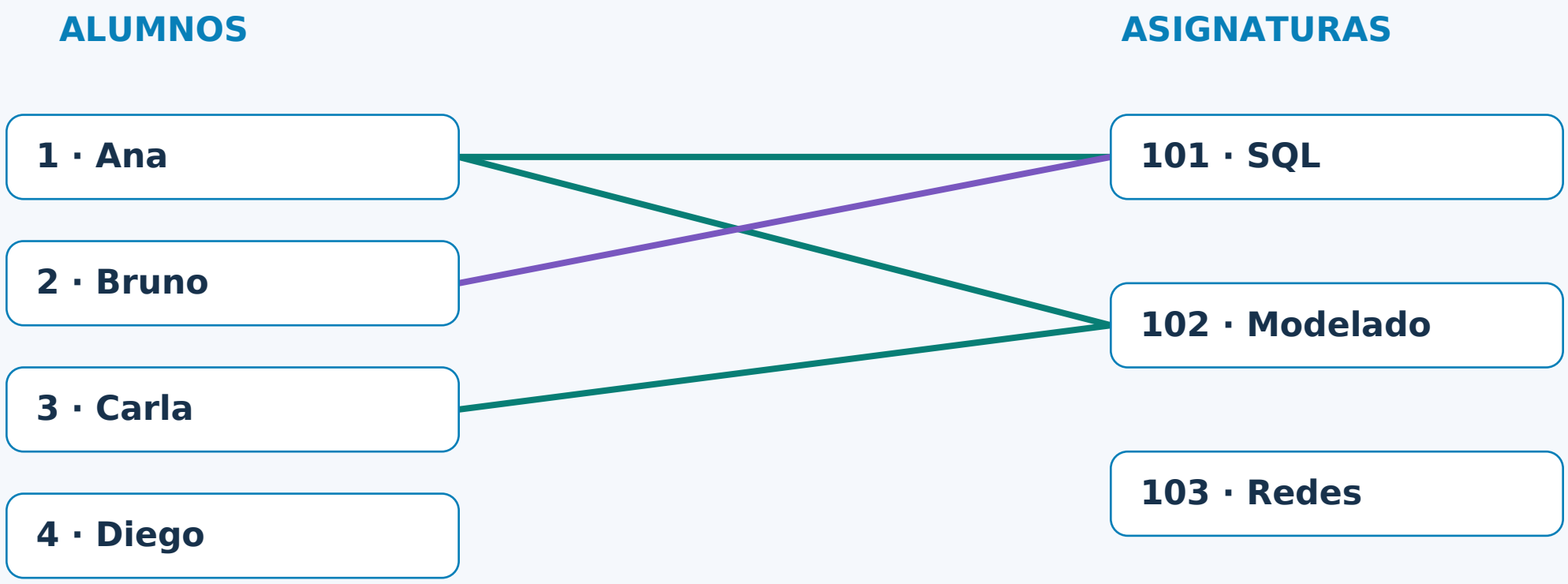
Muchos alumnos, muchas asignaturas

Una relación N:M se entiende mirando en ambas direcciones. N y M indican multiplicidad, no cantidades fijas.



Desde un alumno	Desde una asignatura
Ana cursa SQL y Modelado: varias asignaturas.	SQL tiene a Ana y Bruno: varios alumnos.
Diego no tiene matrículas.	Redes no tiene alumnos.

Cada línea verde o violeta es un hecho

Hay cuatro matrículas. No conectamos automáticamente a todos con todos: con cuatro alumnos y tres asignaturas habría doce parejas posibles, pero solo cuatro existen en nuestro ejemplo.



Una matrícula puede guardar su propia información: nota, fecha de inscripción o estado. La nota de Ana en SQL no tiene por qué coincidir con su nota en Modelado.

Regla de esta guía: una matrícula por pareja alumno-asignatura. Permitimos cero matrículas en ambos extremos. No modelamos cursos académicos ni reintentos todavía.

Dos tablas no guardan las líneas

Tener una lista de personas y otra de materias no indica quién está matriculado en qué.

Intento	Qué problema tiene
asignatura_id dentro de Alumnos	Una sola columna con una FK simple solo permite una asignatura por alumno.
alumno_id dentro de Asignaturas	Solo permite un alumno por asignatura.
asignatura_1, asignatura_2, asignatura_3…	Impone un límite artificial y complica las búsquedas.
Una lista de IDs en una celda	Es una alternativa en modelos documentales, pero no una FK simple que valide cada vínculo.

La solución relacional: una fila por línea

alumno_id	asignatura_id	nota
1 · Ana	101 · SQL	8
1 · Ana	102 · Modelado	9
2 · Bruno	101 · SQL	7
3 · Carla	102 · Modelado	NULL

Esta tabla se llama Matrículas, tabla puente, intermedia o asociativa. Es el registro de los vínculos; no es una copia completa de Alumnos y Asignaturas.

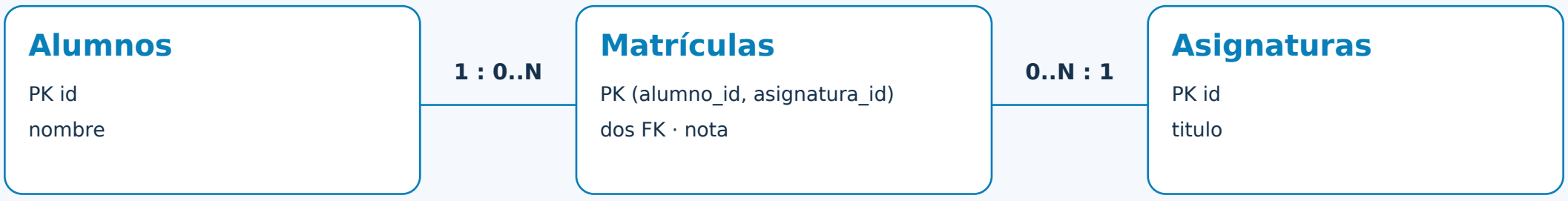
Ana cambia de nombre → UN SOLO SITIO → **Se modifica solo Alumnos**

Ana deja Modelado → EL VÍNCULO → **Se borra solo esa matrícula**

Matiz: una tabla puente es la solución habitual en el modelo relacional normalizado. No es una ley que prohíba arrays o JSON: esos modelos tienen otras reglas de consulta y validación.

Un N:M se convierte en dos 1:N

Cada matrícula pertenece exactamente a un alumno y a una asignatura. Cada extremo puede tener cero o muchas matrículas.



Dos claves foráneas y una clave primaria compuesta

```
CREATE TABLE matriculas (
  alumno_id integer REFERENCES alumnos(id),
  asignatura_id integer REFERENCES asignaturas(id),
  nota numeric(4,2) CHECK (nota BETWEEN 0 AND 10),
  PRIMARY KEY (alumno_id, asignatura_id)
);
```

Regla	Ejemplo
La pareja completa es única	(1,101) no puede insertarse dos veces.
Cada columna puede repetirse	Alumno 1 aparece en dos materias; asignatura 101 tiene dos alumnos.
Las FK comprueban los extremos	(99,101) se rechaza si no existe el alumno 99.
La nota describe a la pareja	Carla está matriculada en Modelado, aunque su nota sea NULL.

PK: identifica una fila e impide NULL en ambas columnas. FK: apunta a una clave válida de la tabla padre. La FK no obliga al padre a tener hijos: Diego y Redes pueden existir sin matrículas.

¿Cuándo cambia esta clave?

Si Ana puede cursar SQL en 2026 y 2027, la pareja ya no basta: podemos incluir curso_academico. Si usamos un matricula_id artificial, seguimos necesitando UNIQUE sobre la combinación que no deba repetirse.

Recorrer el puente en ambos sentidos

No hacen falta siempre dos JOIN. Depende de qué identificadores conoces y qué nombres quieres mostrar.

A · Materias de Ana: conocemos alumno_id = 1

```
SELECT s.titulo
FROM matriculas m
JOIN asignaturas s ON s.id = m.asignatura_id
WHERE m.alumno_id = 1
ORDER BY s.id;
```

Resultado: SQL y Modelado. Un JOIN basta para transformar los IDs de las materias en títulos.

B · Alumnos de SQL: conocemos asignatura_id = 101

```
SELECT a.nombre
FROM matriculas m
JOIN alumnos a ON a.id = m.alumno_id
WHERE m.asignatura_id = 101
ORDER BY a.id;
```

Resultado: Ana y Bruno. El mismo puente funciona en sentido contrario.

C · Todos los vínculos con ambos nombres

```
SELECT a.nombre, s.titulo, m.nota
FROM matriculas m
JOIN alumnos a ON a.id = m.alumno_id
JOIN asignaturas s ON s.id = m.asignatura_id
ORDER BY a.id, s.id;
```

Cuatro filas: Ana/SQL/8, Ana/Modelado/9, Bruno/SQL/7 y Carla/Modelado/NULL. Cada JOIN encuentra una PK única: añade información sin multiplicar cada matrícula.

CROSS JOIN alumnos × asignaturas daría doce parejas posibles. Unir alumnos.id = asignaturas.id tampoco describe quién cursa qué: esa información está en Matrículas.

LEFT JOIN y recuentos correctos

Una fila por matrícula y una fila por alumno son resultados diferentes. Decide cuál necesitas.

```
SELECT a.nombre, s.titulo, m.nota
FROM alumnos a
LEFT JOIN matriculas m ON m.alumno_id = a.id
LEFT JOIN asignaturas s ON s.id = m.asignatura_id
ORDER BY a.id, s.id;
```

alumno	asignatura	nota
Ana	SQL	8
Ana	Modelado	9
Bruno	SQL	7
Carla	Modelado	NULL
Diego	NULL	NULL

Dos LEFT JOIN: si el segundo fuese INNER JOIN, Diego desaparecería. Carla sí tiene matrícula: NULL en nota significa algo diferente a no tener matrícula.

Una fila por alumno: contar sus matrículas

```
SELECT a.nombre, COUNT(m.asignatura_id) AS total
FROM alumnos a
LEFT JOIN matriculas m ON m.alumno_id = a.id
GROUP BY a.id, a.nombre
ORDER BY a.id;
```

Ana 2 · Bruno 1 · Carla 1 · Diego 0. COUNT(columna) no cuenta NULL. COUNT(*) contaría una fila para Diego. LIMIT 1 solo devolvería una fila total.

Alarma: alumnos sin ninguna matrícula

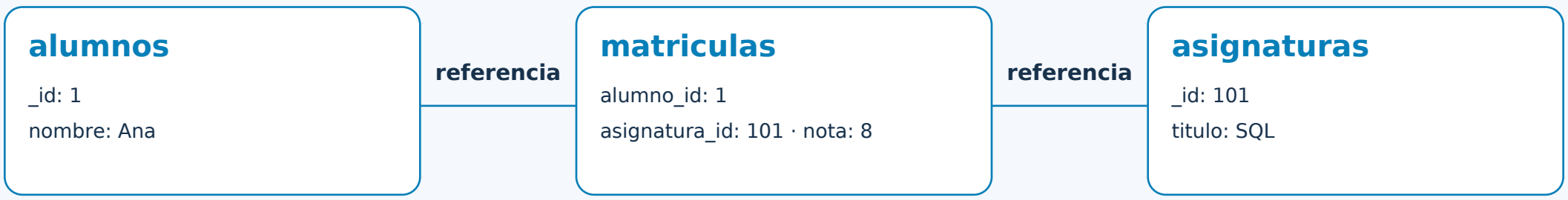
```
SELECT a.nombre
FROM alumnos a
LEFT JOIN matriculas m ON m.alumno_id = a.id
WHERE m.alumno_id IS NULL;
```

Resultado: Diego. Comprueba una clave de la matrícula, no la nota: Carla tiene nota pendiente pero sí está matriculada.

NoSQL también necesita vínculos

Los documentos cambian la representación. No desaparece la pregunta “¿quién cursa qué?”.

A · Una colección de matrículas




```
db.matriculas.createIndex(
  { alumno_id: 1, asignatura_id: 1 },
  { unique: true }
);
```

Una pareja única por documento. Puedes añadir nota, fecha y estado. $lookup reúne documentos de otra colección; devuelve un array. $unwind lo despliega para obtener un resultado por matrícula.

MongoDB no impone una FK relacional entre colecciones. El índice único evita parejas repetidas, pero no verifica que sus extremos existan. Esa integridad requiere lógica explícita.

B · IDs de asignaturas dentro de cada alumno

```
{ _id: 1, nombre: "Ana", asignatura_ids: [101, 102] }
{ _id: 2, nombre: "Bruno", asignatura_ids: [101] }

// Buscar alumnos que cursan SQL:
db.alumnos.find({ asignatura_ids: 101 });
```

Cómodo para leer a una persona con una lista corta y acotada. Un índice sobre el array permite búsquedas por sus elementos. Para mostrar títulos, todavía puedes necesitar $lookup.

Tres colecciones	Array por alumno
Vínculo con identidad y atributos propios; consulta natural desde ambos extremos.	Lectura conjunta del alumno y sus IDs; menos documentos independientes.
Más consultas o agregaciones para reunir nombres.	Hay que cuidar crecimiento, duplicados y actualización de cada elemento.

Guardar solo IDs no copia los títulos. Embeber el título en muchos alumnos sí crea copias que mantener. Evita dos listas espejo sin una estrategia de consistencia. El archivo .js desarrolla ambos modelos.

Del dibujo al código

Los archivos usan los mismos alumnos, materias y cuatro matrículas. Recorre el SQL por bloques, en orden.

Tramo del SQL	Qué observar
00–20 · Crear entidades y vínculos	Por qué la pareja es única y cada ID puede repetirse.
30–40 · Recorrer la relación	Un JOIN cuando ya conoces un extremo; dos para mostrar ambos.
50–60 · Ausencias y resúmenes	LEFT JOIN conserva a Diego. GROUP BY cambia el nivel del resultado.
70–80 · Índices y modificaciones	La FK protege; el índice puede acelerar. Borrar una matrícula no borra al alumno.
90–100 · Vista y evolución	Una vista simplifica consultas; nuevos años o intentos exigen revisar la clave.

¿Qué ocurre con Oracle y MySQL?

Este ejercicio no necesita PL/pgSQL. Tablas, PK, FK y JOIN son conceptos compartidos. El script es PostgreSQL: hay que adaptar esquemas, tipos y transacciones al cambiar de motor; no basta con copiarlo sin revisar.

Motor	Lógica procedural, si llegara a hacer falta
PostgreSQL	PL/pgSQL: DO $$ ... BEGIN ... IF ... END ... $$;
Oracle	PL/SQL: bloques DECLARE / BEGIN / END.
MySQL	IF en programas almacenados; no el mismo DO de PostgreSQL.

En Oracle y MySQL, muchas operaciones DDL producen commits implícitos. Por eso el BEGIN/COMMIT del laboratorio PostgreSQL no se traslada tal cual. CASE dentro de SELECT es distinto de un IF procedural.

Cuatro ideas para recordar

1. Las entidades guardan personas y materias.
2. El puente guarda las parejas que existen.
3. El JOIN añade los datos de cada extremo.
4. En MongoDB, decide dónde guardar esos vínculos y quién verifica su coherencia.

Fuentes oficiales y ejemplos propios:
https://www.postgresql.org/docs/current/ddl-constraints.html
https://www.postgresql.org/docs/current/queries-table-expressions.html
https://www.mongodb.com/docs/manual/data-modeling/
https://www.mongodb.com/docs/manual/tutorial/model-embedded-many-to-many-relationships-between-documents/
Sintaxis SQL revisada; ejemplos no ejecutados contra servidores PostgreSQL o MongoDB en este entorno.